

Unit

3

Lesson

2



Beyond Imagination. Connected & Protected.

Time Frame: [90 min]

Lesson Objectives:

By the end of this lesson students will be able to:

- Write code to implement the desired app functionality using App Inventor.
- Understand the purpose of a notifier in an app (e.g., displaying alerts or messages to users).

Challenge questions of the lesson

- How can adding a notifier improve the user experience in your app? Give a real-life example.
- What steps would you take in App Inventor to make a button display a warning message when pressed?
- Why might an engineer choose to use a notifier instead of changing text on the screen?
- Beyond emergency preparedness, how can the principles of multi-sensory feedback and pattern recognition in smart systems be applied to solve other complex social or environmental challenges within a community?

SEL Relation

- **Social Awareness:** Cultivating profound empathy by designing technology-driven solutions that address critical community safety needs and cater to the diverse requirements of all individuals.
- **Responsible Decision-Making:** Grappling with the ethical implications, reliability considerations, and potential biases inherent in creating autonomous smart alert systems, especially for critical emergency responses.
- **Relationship Skills:** Engaging in complex collaborative design and problem-solving, practicing constructive critique and effective communication within a team setting.
- **Self-Management:** Demonstrating resilience and perseverance in tackling highly complex coding and system integration challenges, particularly when implementing pattern recognition algorithms.
- **Self-Awareness:** Reflecting on their evolving capabilities as engineers and designers in addressing significant societal challenges through technology.

Engage



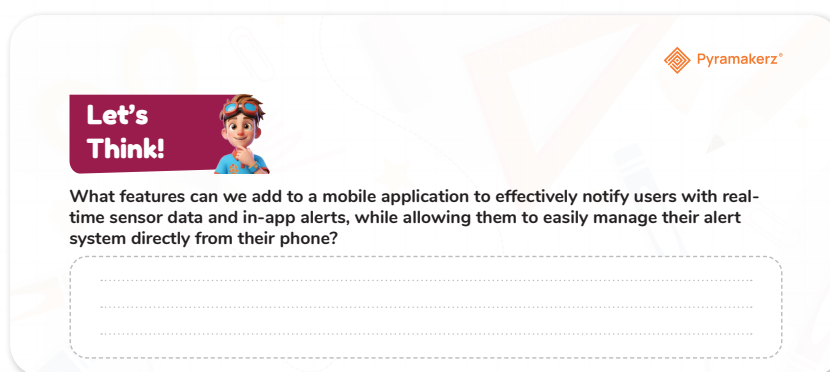
Teacher's Role:

- Introduce the concept of mobile app notifications by relating it to common experiences (e.g., weather alerts, social media notifications, or reminder apps).
- Present the Let's Think question visually and read it aloud to the class.
- Encourage brainstorming by prompting students to think about what makes alerts helpful versus annoying.
- Facilitate a short discussion about real-time data and user control in apps.
- Provide vocabulary support for terms like real-time, in-app alert, sensor data, and user interface.

Expected from Students:

- Reflect on personal experiences with notifications in apps they use.
- Suggest features that would make alerts more useful, timely, and easy to manage.
- Share ideas on how mobile apps can give users control over the type, timing, and format of alerts.
- Collaborate in pairs or small groups to expand on each other's suggestions.

Parts of the book included.



Let's Think!



Objective:

Students will generate and evaluate ideas for app features that combine real-time data, notifications, and user control. This primes them for designing and coding their own notifier feature in MIT App Inventor.

Answers:

Possible Student Responses:

- Adjustable notification settings (sound, vibration, pop-up, silent mode).
- Alerts that show sensor readings (e.g., temperature, water level, air quality) directly in the message.
- An alert history log to view past notifications.
- Ability to mute or snooze alerts for a set period.
- Customizable trigger thresholds (e.g., alert when temperature exceeds 30°C).
- Push notifications even when the app is closed.

Possible Strategies for Implementation:

1. Think–Pair–Share

- First, have students write their ideas individually.
- Then pair up to exchange ideas.
- Finally, share the most promising features with the whole class.

2. Real-World Example Analysis

- Show short screenshots or demo videos of notification settings in common apps (weather app, home security app).
- Ask students to identify which features make these apps effective.

3. Feature Sorting Activity

- Prepare a list of potential notification features (e.g., “vibration only,” “voice alert,” “changeable threshold,” “no user control”).
- Have students sort them into “must-have,” “nice-to-have,” and “not necessary.”

4. Problem–Solution Mapping

- On the board, write a few real-world scenarios (e.g., “water sensor detects leak,” “air quality drops suddenly”).
- Students suggest what type of alert would be best in each case.

5. Future Feature Brainstorm

- Challenge students to imagine what app notifications might look like in 10 years and how these could help users manage real-time sensor systems.

Explorer's Toolkit



Teacher's Role:

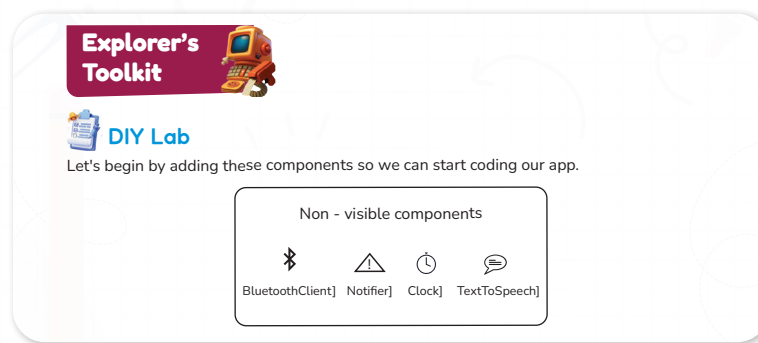
- Guide students in identifying and adding the necessary non-visible components for their app in MIT App Inventor.
- Model each step on a projected screen or large display so students can follow along in real time.

- Encourage students to understand the purpose of each component, not just add it blindly.
- Provide immediate troubleshooting support if students cannot locate or place components.
- Reinforce the connection between these components and the app's functionality for notifications, time-based triggers, and Bluetooth communication.

Expected from Students:

- Open MIT App Inventor on their devices and navigate to the correct project or create a new one.
- Locate and drag the required components from the Palette to the Viewer.
- Place the components in the Non-visible Components area.
- Save their work after each step.
- Be prepared to explain what each component does.

Parts of the book included.



DIY Lab

Objective:

Students will add and configure the non-visible components BluetoothClient, Notifier, Clock1, and TextToSpeech in MIT App Inventor to prepare for coding the app's real-time notification system.

Required Materials:

- Computer or tablet with internet access
- MIT App Inventor account (logged in)
- Project workspace opened (new or existing project)
- Teacher's demonstration device with projector or screen-sharing

Activity Instructions:

Step 1 – Open Your Project

1. Log in to MIT App Inventor.
2. From the Projects menu, select your existing app project or create a new one by clicking Start New Project and naming it (e.g., NotifierApp).

3. Wait for the workspace to load fully.

Step 2 – Locate the Palette

1. On the left-hand side of the screen, find the Palette panel.
2. Familiarize yourself with the categories such as User Interface, Sensors, Connectivity, and Media.

Step 3 – Add the BluetoothClient Component

1. Click on the Connectivity category.
2. Locate BluetoothClient.
3. Click and drag BluetoothClient into the Viewer.
4. It will appear in the Non-visible Components section at the bottom of the screen.

Step 4 – Add the Notifier Component

1. Click on the User Interface category in the Palette.
2. Scroll down to find Notifier.
3. Drag Notifier into the Viewer.
4. Check that it appears in the Non-visible Components section.

Step 5 – Add the Clock Component

1. Click on the Sensors category in the Palette.
2. Locate Clock.
3. Drag Clock into the Viewer.
4. Confirm it is now listed under Non-visible Components.

Step 6 – Add the TextToSpeech Component

1. Click on the Media category in the Palette.
2. Locate TextToSpeech.
3. Drag it into the Viewer.
4. Verify that it is in the Non-visible Components section.

Step 7 – Save Your Work

1. Go to the Projects menu and select Save project.
2. Wait for the confirmation message that your project has been saved successfully.

Possible Strategies for Implementation:

1. Live Modeling with Pauses

- Demonstrate each component addition slowly, pausing after each step so students can catch up.

- Ask “What does this component do?” before moving on to the next one.

2. Think-Aloud Method

- Explain your reasoning as you add each component. For example: “We’re adding the Clock so we can check the time and trigger alerts at specific intervals.”

3. Peer Coaching

- Pair students so that one acts as the “navigator” (reading the steps) and the other as the “driver” (doing the work).
- Switch roles halfway through.

4. Component Purpose Cards

- Prepare small cards with the names and functions of each component.
- After adding them, students match each card to the correct component in their project.

5. Quick Check Challenge

- Before saving, ask students to list aloud the four components they have just added and one way each will be used in their app.

Explain



Teacher's Role:

- Introduce the focus of this lesson segment: learning how to add and configure the Notifier component in MIT App Inventor.
- Present the video link and explain that it will demonstrate practical steps and examples for using the Notifier.
- Set a clear learning purpose before watching so students know what to look for.
- Pause at key moments in the video to clarify terminology and demonstrate important steps live if needed.
- Facilitate a discussion afterward to ensure understanding of how and when to use notifications in an app.

Expected from Students:

- Watch the video attentively, noting down important steps for adding and configuring a Notifier.
- Identify the different types of Notifier messages (alerts, choose dialog, text input dialog).
- Recognize situations where a Notifier would be beneficial in an app.
- Engage in post-viewing discussion and ask questions if unclear about implementation.

Parts of the book included.



Screen Lessons

Click on the link to learn how can we add and use notifier in MIT app inventor.

<https://youtu.be/2Ft1-SeWXkA?si=B1pj5ljQLYJ2eqFS>



Screen Lessons

Objective:

Students will understand how to add, configure, and use the Notifier component in MIT App Inventor to display alerts and messages to app users, enhancing app functionality and user interaction.

Video Link:

<https://youtu.be/2Ft1-SeWXkA?si=KK8JOAq5FOmkRnlq>



Possible Strategies for Implementation:

1. Pre-Watching: Set a Purpose

Strategy: Guided Viewing Questions

Implementation:

- Write these questions on the board:
 1. How do you add the Notifier component to your app?
 2. What different types of Notifier messages are available?
 3. When would you use each type of message?
- Tell students to find answers to these questions during the video.

2. During Watching: Active Engagement

Strategy: Pause & Practice

Implementation:

- Play the video in short segments.
- After the part showing how to add a Notifier, pause and have students add the component in their own app.
- Pause again when different Notifier blocks are introduced. Ask students to predict what the block will do before the video shows the result.

3. Post-Watching: Discussion & Reflection

Strategy: Concept Application Discussion

Implementation:

- Lead a whole-class discussion:
 - “What is the difference between a simple alert and a choose dialog?”
 - “How could we use a Notifier in our ECO Boat app to alert users about sensor data?”

- Ask students to open their app project and create a simple Notifier message that says: "Test Alert – System is Active".
- Have volunteers share their screens to show their Notifier in action.

Elaborate and Evaluate



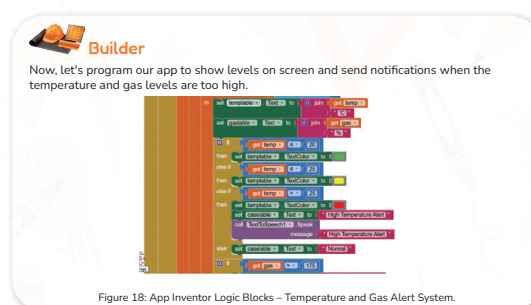
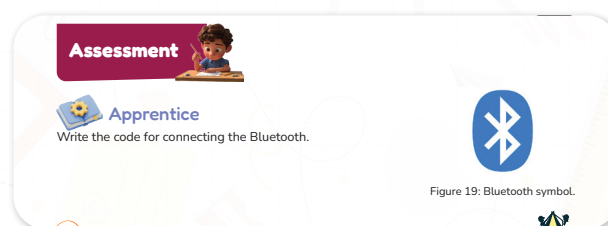
Teacher's Role:

- Guide students in applying their knowledge of Bluetooth connectivity and Notifier functions to build a fully functional alert system in their app.
- Provide step-by-step guidance for the Apprentice task (connecting Bluetooth) before moving to the more complex Builder task (programming sensor-based notifications).
- Model the logic and code structure visually using the MIT App Inventor Blocks Editor.
- Encourage students to test each section of their code as they go to ensure correct functionality.
- Offer troubleshooting tips for common errors such as missing permissions, incorrect device pairing, or misplaced code blocks.

Expected from Students:

- Successfully create and arrange the code blocks needed to connect their app to a Bluetooth device.
- Program the app to display temperature and gas readings, change text colors based on conditions, and trigger alerts when values exceed set thresholds.
- Test their app with simulated or real sensor data to confirm that notifications work correctly.
- Make adjustments to thresholds or alert messages as needed.

Parts of the book included.





Apprentice

Objective:

Students will write the code necessary to establish a Bluetooth connection between the app and a Bluetooth-enabled microcontroller.

Materials:

- Computer with MIT App Inventor access
- Mobile device with MIT App Inventor Companion app installed
- Bluetooth-enabled microcontroller (e.g., Arduino with HC-05 module)
- USB cable for programming
- Internet connection

Activity Instructions:

1. Open the MIT App Inventor Project

Load the app project that will connect to the Bluetooth device.

2. Add Bluetooth Client Component

In the “Connectivity” palette, drag the BluetoothClient component to the app workspace.

3. Create Connection Block

Program a button (e.g., “Connect”) to open a list of available Bluetooth devices.

4. Select Device

Ensure the block connects to the selected Bluetooth module’s MAC address.

5. Test the Connection

Pair the mobile device with the module in phone settings, then test the connection button.

Answer Guide:

Students should produce a working block set that connects the app to a Bluetooth device using the BluetoothClient.Connect block.

Code:

```

when ListPicker1 .BeforePicking
do
  set ListPicker1 . Elements to BluetoothClient1 . AddressesAndNames

when ListPicker1 .AfterPicking
do
  if
    call BluetoothClient1 .Connect
    address ListPicker1 . Selection
  then
    set ListPicker1 . Elements to BluetoothClient1 . AddressesAndNames

when Screen1 .Initialize
do
  if
    not BluetoothClient1 . Enabled
  then
    call Notifier1 .ShowAlert
    notice "Bluetooth is not Enabled, go Settings to enable"
  
```

Possible Strategies for Implementation:

- **Model First:** Demonstrate how to pair the mobile device with the Bluetooth module before coding.
- **Connection Troubleshooting:** Provide a checklist (check power, wiring, pairing status).
- **Peer Debugging:** Have students swap projects to check if the code connects on another device.
- **Small Wins Approach:** Test Bluetooth connection before adding any sensor logic.
- **Connection Reflection:** Ask, "Why might the connection fail, and how could you handle that in code?"



Builder

Objective:

Students will program the app to display temperature and gas readings, change display colors based on threshold levels, and trigger alerts.

AIA file:

<https://drive.google.com/file/d/1ExkjcqbySoZHvqHukl9jJkFD1M3doafg/view?usp=sharing>



Materials:

- Completed Apprentice task app with Bluetooth connection
- Temperature and gas sensors connected to the microcontroller
- Internet-enabled computer with MIT App Inventor

Step-by-Step Instructions

1. Receive Data from Bluetooth

Use the `BluetoothClient.ReceiveText` block to capture incoming temperature and gas data.

2. Separate Sensor Values

Parse the incoming text into two variables: temperature and gas level.

3. Display Sensor Readings

- Set the temperature label to show the temperature value.
- Set the gas label to show the gas value.

4. Apply Conditional Logic

- If temperature > 28°C → set text color to red, display “High Temperature Alert,” and use Text-to-Speech to announce it.
- If gas > 175 ppm → set text color to yellow, display “Gas Alert.”

5. Else Conditions

If neither threshold is exceeded, show “Normal” with default colors.

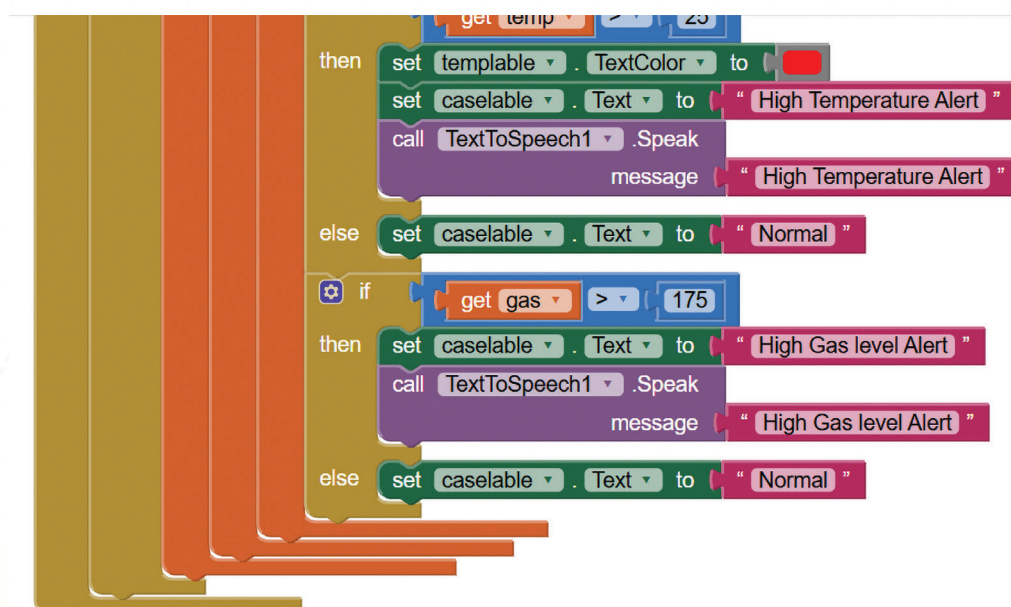
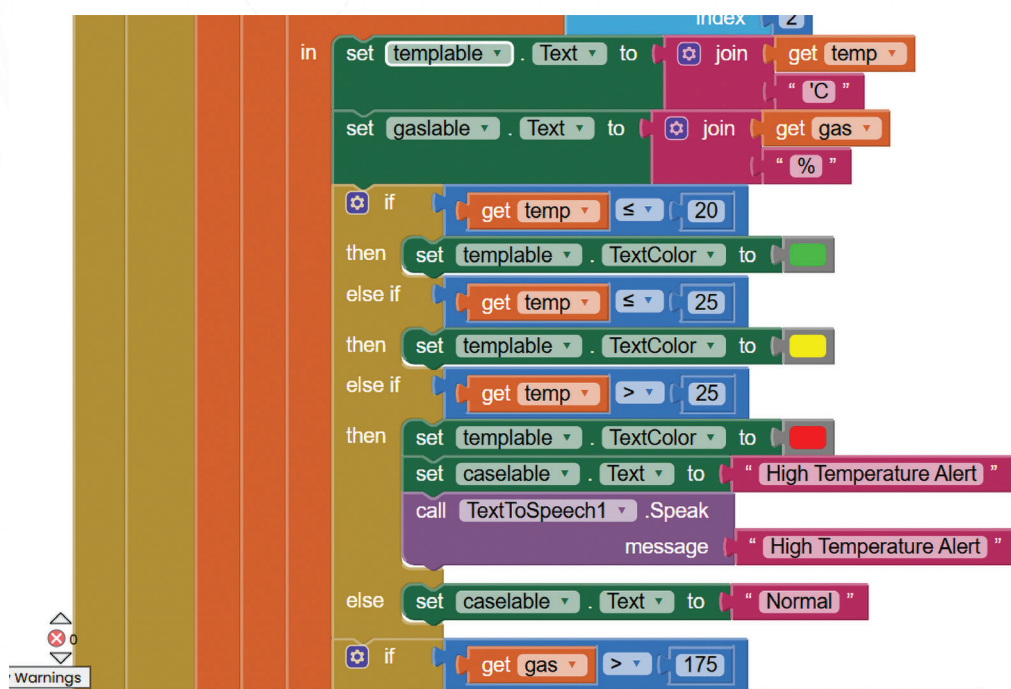
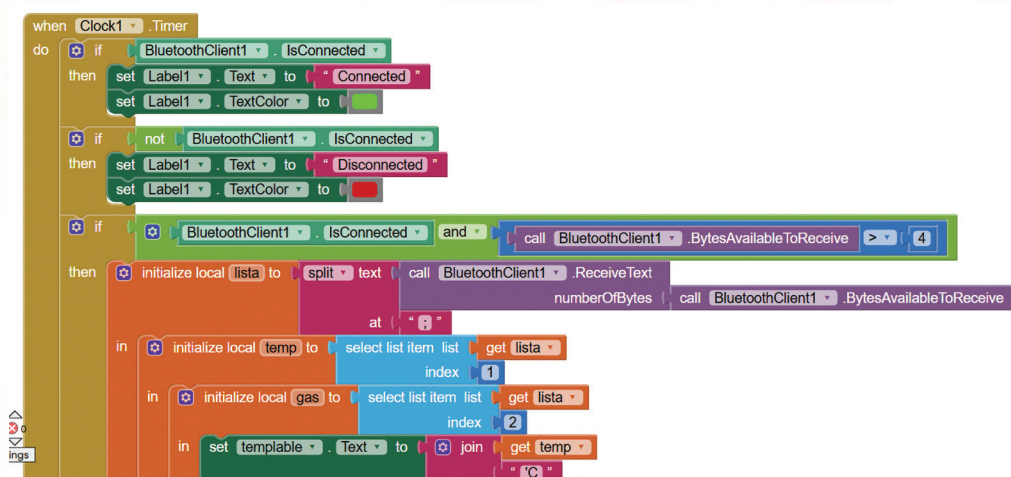
6. Test the Program

Simulate or use live sensor data to ensure alerts function correctly.

Code:

```
when BluetoothClient1 .BluetoothError
  functionName message
  do
    call BluetoothClient1 .Disconnect
    if not BluetoothClient1 .IsConnected
    then
      set Label1 .Text to "Disconnected"
      set Label1 .TextColor to red
      call TextToSpeech1 .Speak
      message "Disconnected"

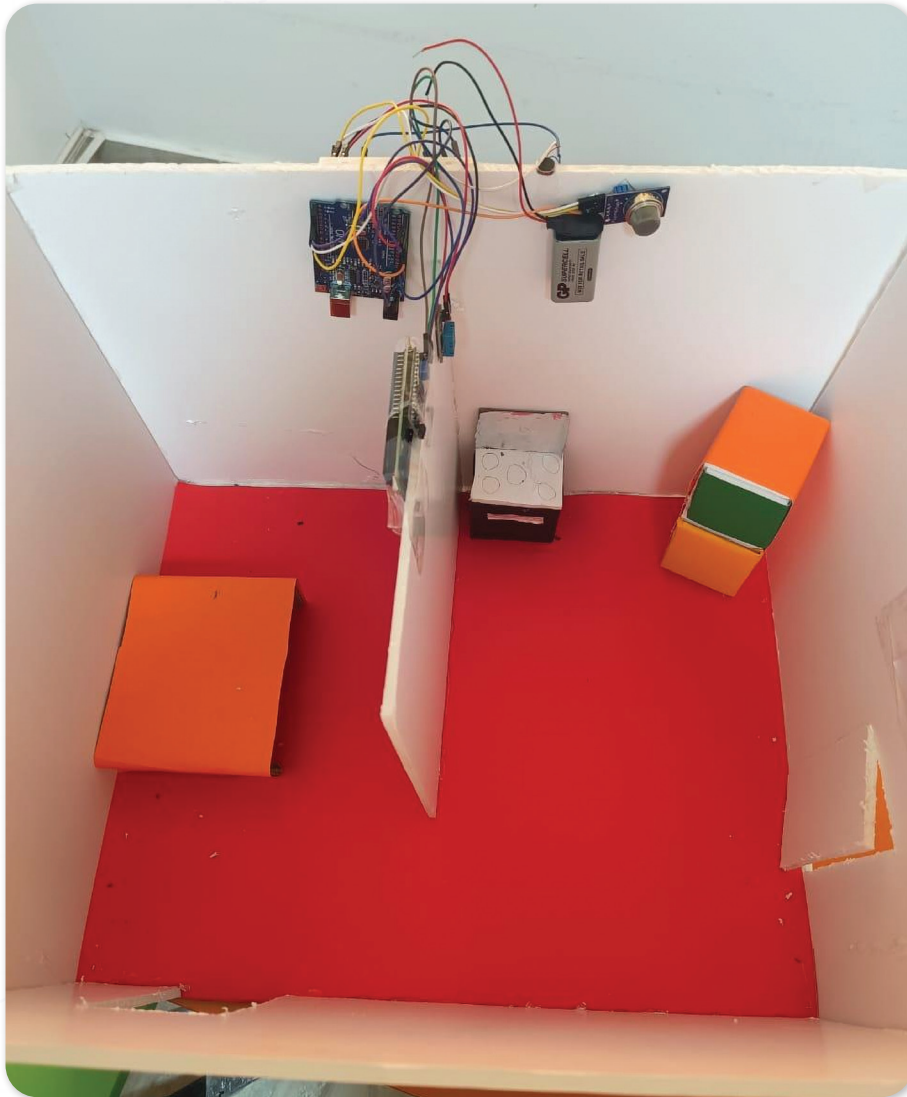
when BtnDisconnect .Click
  do
    call BluetoothClient1 .Disconnect
    call TextToSpeech1 .Speak
    message "Disconnected"
```



Possible Strategies for Implementation:

- Think-Aloud Coding: Teacher explains each block's purpose while building the code live.
- Threshold Discussion: Let students decide on safe values for alerts and justify choices.
- Pair Testing: Have students test each other's apps with mock sensor values.
- Color Feedback: Discuss how color changes can make an interface more intuitive.
- Alert Enhancement: Encourage students to add vibration or sound for stronger warnings.
- warnings.

Final Prototype:

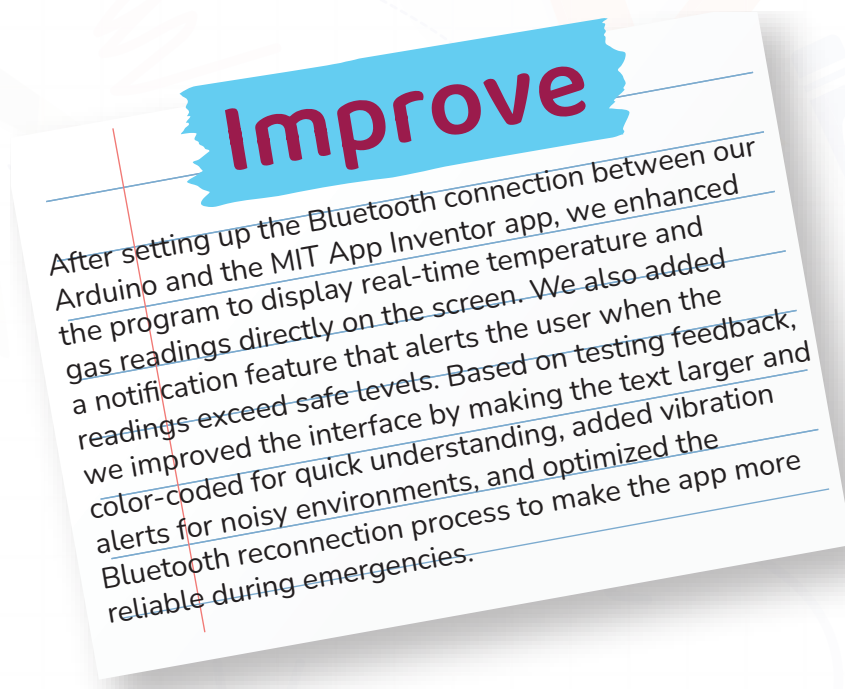


Final prototype video:

https://drive.google.com/drive/folders/12DtkwijgRObjTY5cfcXsmI9Cp1rgqaIT?usp=drive_link



Don't forget to go back to EDP paper and add this to improve part.



Now I Can

Objective:

Help students reflect on their learning by connecting the project to real-world applications and personal growth.

Activity Instructions:

Guide students through reflection using this three-part structure:

Step 1: Revisit the Lesson's Purpose

- Ask: "What was the big idea or problem we worked on?"
- Record responses together to build a shared understanding

Step 2: Student Self-Reflection

- Students complete the sentence:

"Now I can..."

- Encourage specific references to tools, skills, or concepts
- Example:

"Now I can design an app that alerts users based on real-time sensor data."

Step 3: Small Group Discussion

- Use a Kagan routine like Timed Pair Share or RallyRobin
- Students share their reflections in pairs or groups
- Encourage comments like:
 - "What's something new you learned from your partner?"

- o “What would you like to try differently next time?”

STANDARDS ALIGNMENT

This lesson connects to several important educational standards related to technology and engineering. Specifically, it aligns with the following frameworks:

1. NGSS (Next Generation Science Standards)

- **HS-ETS1-3:** Design a solution to a complex real-world problem.
 - o Students design a mobile application that uses real-time sensor data to notify users and manage alert systems. This directly relates to problem-solving and design, aligning with the standards for high school engineering practices.

2. ISTE (International Society for Technology in Education) Standards

- **Empowered Learner:** Students leverage technology to take an active role in choosing, achieving, and demonstrating competency in their learning goals.
 - o Through the creation of an app, students learn to use technology tools to enhance their understanding of sensors and alert systems. The lesson encourages self-directed learning by allowing students to manage their projects and develop a functional app.
- **Computational Thinker:** Students develop and employ strategies for understanding and solving problems using computational thinking techniques.
 - o The lesson promotes problem-solving skills through coding and applying logical thinking to build a real-world application, which requires a deep understanding of computational concepts.

3. Common Core State Standards for Mathematics (CCSSM)

- **Math.8.SP.A.1:** Construct and interpret scatter plots.
 - o In the context of working with sensor data, students may analyze patterns in the data (such as temperature or gas levels) and represent this information graphically.

4. Local/State Technology Standards

- **Design and Technology:** Students apply design and technology processes to create innovative solutions.
 - o The project aligns with state-level technology standards that focus on the design process, programming, and applying technological solutions to real-world problems.



Mission Innovate

Final Mission – Design for Safety and Security

Overview:

In this open-ended project, students will design a tech-based solution to help individuals with disabilities feel safer and more secure in their homes—especially when they may be alone or in vulnerable situations. The goal is to create a tool, device, or system that can detect potential threats, alert the user, and notify family, caregivers, or authorities. Students

will apply coding, electronics, and design thinking to develop an inclusive safety prototype.

Project duration: 2 weeks

Teacher's Role:

- **Facilitator of Inquiry**
 - Begin by reading the mission prompt aloud. Spark curiosity by asking:
“What safety concerns might people with disabilities face when they are alone at home?”
“What could we design to help them feel secure and able to alert others in an emergency?”
 - Support students as they brainstorm real-life safety challenges and possible tech-based solutions.
- **Encourager of Empathy and Design Thinking**
 - Guide students to research existing tools such as panic buttons, motion sensors, and smart alarms.
 - Encourage them to consider different types of alerts (sound, light, vibration, notifications) to suit various disabilities.
- **Supporter of Student-Led Innovation**
 - Allow students to choose a problem to solve and decide on the essential features for their design.
 - Provide flexibility in prototyping—students may use MIT App Inventor, mBlock, Arduino, PictoBlox, or recycled materials.
- **Guide for Reflection**
 - Prompt students to reflect on:
“How does your design improve personal safety for individuals with disabilities?”
“Which features make your solution accessible to different types of users?”

Assessment Guide

Use a rubric to evaluate:

- Clear identification of the safety problem and target user needs
- Creativity and feasibility of the solution
- Functionality and relevance of prototype features

Clarity and empathy in presentation of the idea

Collaboration and problem-solving effort

Implementation Tips:

- Allow students to work individually or in teams for idea sharing and peer support.
- Provide access to:
 - Craft materials (cardboard, paper, LEDs, buzzers, vibration motors)
 - Computers or tablets with coding/design software

- Internet access for researching safety technologies
- Encourage students to test their design with a peer acting as the “end user” to evaluate usability and response time.
- Dedicate time for students to present their work through a gallery walk, live demo, or video pitch.